

Copy Constructor

Yes — this is one of the **most important C++ OOP interview areas**. Let's make the distinction very clear.

1. When is a Copy Constructor Called?

A copy constructor is called when a **new object is created from an existing object**.

Typical form:

```
ClassName(const ClassName& other);
```

Example:

```
class Student {
public:
    int age;

    Student(int a) : age(a) {}

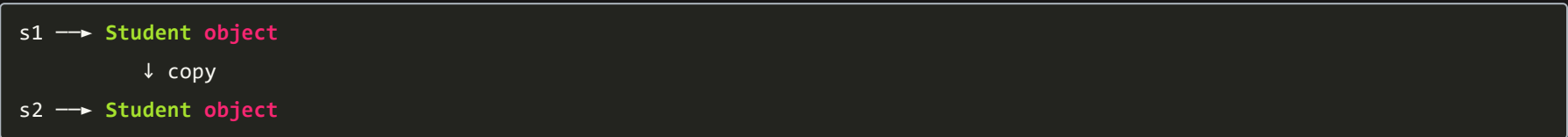
    Student(const Student& other) {
        age = other.age;
    }
};
```

Now let's see the important cases.

Case 1: Direct initialization

```
Student s1(20);
Student s2(s1);
```

s2 is a **new object**, so copy constructor is called.



Case 2: Copy initialization

```
Student s1(20);
Student s2 = s1;
```

Copy constructor is also used here.

Interview trap

This:

```
Student s2 = s1;
```

is **not copy assignment**.

It's initialization of a new object.

Compare:

```
Student s2 = s1;    // Copy constructor
```

vs

```
Student s2;
s2 = s1;           // Copy assignment operator
```

Easy rule

- New object → copy constructor
- Existing object → assignment operator

2. Passing Object by Value

```
void print(Student s) {  
    // ...  
}  
  
Student s1(20);  
print(s1);
```

The parameter:

```
Student s
```

is a new object initialized from `s1`.

Therefore, a copy may be made using the copy constructor.

If you instead do:

```
void print(const Student& s)
```

no copy is needed.

Interview tip

This is why objects are commonly passed as:

```
void func(const Student& s)
```

instead of:

```
void func(Student s)
```

when you don't need a copy.

3. Returning an Object by Value

Consider:

```
Student createStudent() {  
    Student s(20);  
    return s;  
}
```

Historically, returning by value could involve copying `s`.

But **modern C++ uses copy elision extensively**, and the compiler can construct the returned object directly.

So don't say in an interview:

```
"Returning by value always calls the copy constructor."
```

That's **not correct**.

Better answer:

```
"Returning an object by value may involve copy/move construction, but modern C++ often eliminates those operations through copy elision."
```

4. Shallow Copy vs Deep Copy

This is where copy constructors become especially important.

Consider:

```
class Student {  
public:  
    int* age;  
  
    Student(int a) {  
        age = new int(a);  
    }  
};
```

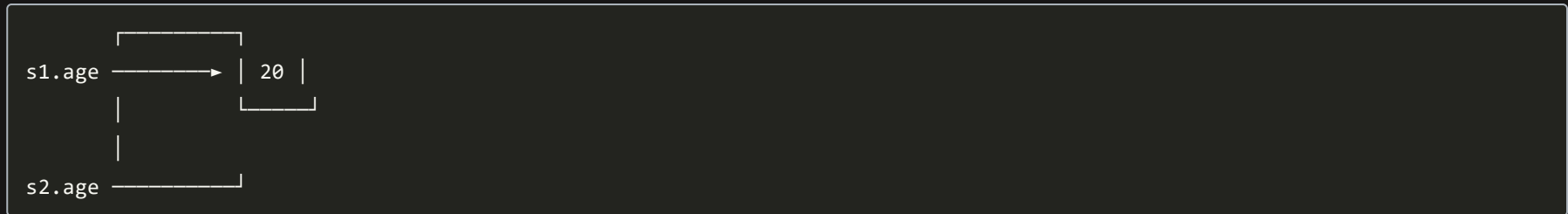
Create:

```
Student s1(20);  
Student s2 = s1;
```

The compiler-generated copy constructor essentially performs:

```
s2.age = s1.age;
```

So:



Both objects point to the **same memory**.

That's **shallow copy**.

5. Why Shallow Copy Is Dangerous

Suppose the class has a destructor:

```
class Student {
public:
    int* age;

    Student(int a) {
        age = new int(a);
    }

    ~Student() {
        delete age;
    }
};
```

Now:

```
Student s1(20);
Student s2 = s1;
```

Both `s1.age` and `s2.age` point to the same memory.

When `s2` is destroyed:

```
delete age
```

Memory is freed.

Then when `s1` is destroyed:

```
delete age
```

It tries to free the **same memory again**.

That's **double deletion / undefined behavior**.

6. Deep Copy

Deep copy means:

Create a completely independent copy of dynamically owned resources.

We write our own copy constructor:

```
class Student {
public:
    int* age;

    Student(int a) {
        age = new int(a);
    }

    Student(const Student& other) {
        age = new int(*other.age);
    }

    ~Student() {
        delete age;
    }
};
```

Now:

```
Student s1(20);
Student s2 = s1;
```

Memory looks like:

```
s1.age —→ [20]

s2.age —→ [20]
```

Two different memory locations.

So:

```
*s2.age = 30;
```

doesn't affect:

```
*s1.age
```

7. The Key Difference

	Shallow Copy	Deep Copy
Copies the pointer/address	Yes	No
Copies the actual resource	No	Yes
Both objects may share resource	Yes	No
Objects have independent resources	No	Yes
Usually compiler-generated member-wise copy	Yes	No
Usually requires custom copy logic for raw owning pointers	No	Yes
Can cause double deletion	Yes	No
Avoids shared ownership problems	No	Yes

Visualize it:

Shallow

```
Object A —┐
             └─→ [Resource]
Object B —┐
```

Deep

```
Object A —→ [Resource A]

Object B —→ [Resource B]
```

8. Very Important: Copy Constructor Doesn't Always Mean Deep Copy

This is a common misconception.

A copy constructor can perform either:

```
Student(const Student& other) {
    // shallow copy
}
```

or:

```
Student(const Student& other) {
    // deep copy
}
```

The **copy constructor** is simply the mechanism for constructing one object from another.

Whether the copy is shallow or deep depends on what it does.

9. What Does the Compiler-Generated Copy Constructor Do?

Suppose:

```
class Student {
    int age;
    string name;
};
```

The compiler-generated copy constructor performs **member-wise copying**.

Conceptually:

```
Student(const Student& other)
    : age(other.age),
      name(other.name) {}
```

For normal value-type members like `int`, that's fine.

For a raw pointer:

```
int* age;
```

member-wise copying copies the **pointer**, not the pointed-to object.

That's why raw owning pointers create the shallow-copy problem.

10. Interview Question: What About `std::string`?

Suppose:

```
class Student {  
    string name;  
};
```

If:

```
Student s2 = s1;
```

you don't need to manually implement a deep-copy constructor just because `string` manages memory internally.

`std::string` already manages its own resources correctly.

This is one reason modern C++ prefers **RAII/resource-managing types** like:

```
std::string  
std::vector  
std::unique_ptr  
std::shared_ptr
```

over manually managing raw `new`/`delete`.

Most Important Interview Question

Q: When is the copy constructor called?

A strong interview answer:

"The copy constructor is used when a new object is initialized from an existing object of the same class. Common cases are direct initialization like `A b(a)`, copy initialization like `A b = a`, and passing an object by value. Returning by value may also involve copy or move construction, although modern C++ often eliminates the operation through copy elision."

Q: What is shallow copy?

"Shallow copy copies members as-is. For a pointer member, this means copying the address, so multiple objects can point to the same resource."

Q: What is deep copy?

"Deep copy creates an independent copy of the resource, so the copied objects don't share the same dynamically allocated memory."

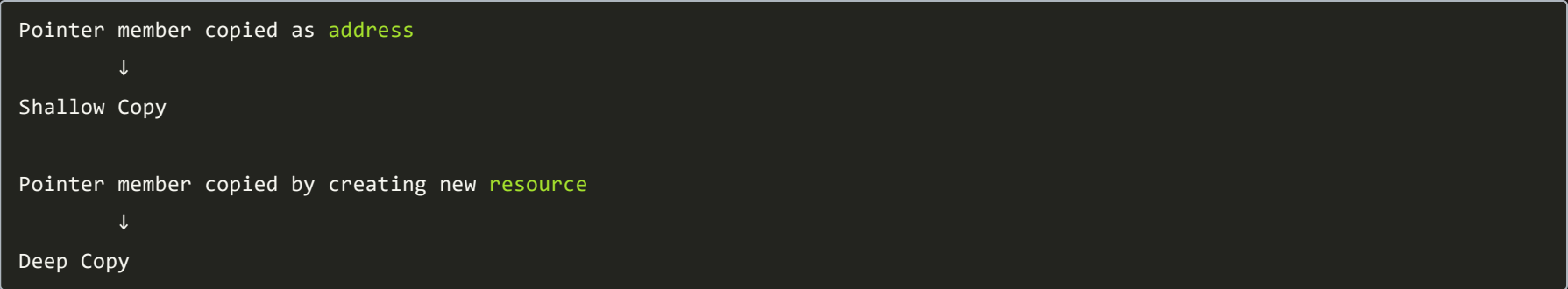
Q: Why do we need a custom copy constructor?

"When the class owns a resource that cannot safely be copied by simple member-wise copying, such as dynamically allocated memory managed through a raw pointer."

One Rule to Remember

```
NEW object from OLD object  
    ↓  
Copy Constructor  
  
OLD object = OLD object  
    ↓  
Copy Assignment Operator
```

And:



The **next interview-level concept** after this is **Rule of 3** → **Rule of 5** → **Rule of 0**, because deep-copying a raw pointer naturally leads directly into copy constructor + copy assignment + destructor and then move semantics.